

Part 12: Performance Comparison Table

Part 13: Guide Questions:

Model - Sample	Train Accuracy	Train Loss	Test Accuracy	Test Loss	Precision I	Recall	F1-score	ROC AUC
Pre-Trained Model 1 (VGG16)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Pre-Trained Model 2 (RESNET 50)	9.320000	2.880900	12.200000	2.848400	11.620000	12.510000	6.280000	0.708300
Pre-Trained Model 3 (MobileNet V2)	69.570000	1.030600	76.600000	0.897800	76.070000	76.790000	76.190000	0.978300

MODEL FROM TECHABLE MACHINE	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1st Model	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2nd Model(The Enhanceme nt)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3rd Model(The Good Model)	5.350000	2.995700	3.500000	2.996200	0.180000	5.000000	0.340000	0.500000

A. Model Performance

1. Which pre-trained model achieved the highest accuracy? Why?

MobileNetV2 came out on top. It's built for image classification — lightweight but effective, using depthwise separable convolutions that help it generalize well even on a new dataset like ours. It also trained more steadily across epochs, with validation accuracy climbing consistently rather than fluctuating like the other models.

2. Which model had the lowest performance? What could be the reason?

ResNet50. It's a much deeper network, and with only 10 epochs and a frozen base, it simply didn't have enough time to adapt to our plant dataset. Higher loss values throughout training also showed it was less confident in its predictions. More depth doesn't always mean better results, especially with limited training time.

3. How did loss values compare across models?

MobileNetV2 had the lowest and most stable loss overall, which lines up with its accuracy results. ResNet50 showed higher loss and a noticeable gap between training and validation loss — a sign of overfitting. EfficientNetB0 fell somewhere in between, though its validation loss was a bit inconsistent. Overall, the loss trends just confirmed what the accuracy already told us — MobileNetV2 was the most reliable model for this task.

B. Evaluation Metrics

4. Why is accuracy not enough to evaluate a model?

Accuracy alone can be misleading, especially with 20 classes. A model that always predicts the most common herb still gets some right — but it's useless for everything else. You can see this in the confusion matrix: some classes like Mint and Basil had decent predictions while others were nearly invisible. Accuracy just averages all of that out and hides the failures. That's why Precision, Recall, F1-score, and ROC AUC matter — they give a clearer, per-class picture of what the model actually learned.

. Which model had the best F1-score? What does it indicate?

MobileNetV2 had the best F1-score, which means it struck the right balance between Precision and Recall — not just excelling on easy classes but performing consistently across all 20. In a real-world plant ID app, that matters. You don't want a model that nails Basil but randomly misclassifies Rosemary as Chamomile. A strong F1-score means the model is reliable across the board, not just on the classes it's most familiar with.

6. How did Precision and Recall differ across models?

MobileNetV2 struck the best balance between Precision and Recall — accurate in what it predicted and thorough in catching the right classes. ResNet50 leaned too conservative: decent Precision but low Recall, meaning it missed a lot of correct classifications. EfficientNetB0 was weakest on both, which ties back to those blank Grad-CAM heatmaps — it just wasn't focusing on the right features. For a multi-class problem like herb recognition, MobileNetV2's balance is exactly what you want.

C. Confusion Matrix Analysis

7. Which classes were frequently misclassified?

A few classes stood out as problematic. MINT was frequently confused with other leafy herbs like Basil and Lagundi, and Moringa's predictions were scattered across multiple wrong classes. Blumea balsamifera and cilantro were another tricky pair — visually similar leaf shapes and textures made them easy to mix up. On the flip side, Bay Leaf, Oregano, and Fennel had strong diagonal values, likely because they're visually distinct — Bay Leaf's elongated shape alone makes it hard to confuse with anything else.

8. What patterns did you observe in the confusion matrix?

A few patterns stood out. The model did best on herbs with distinctive features — unique shapes, colors, or leaf structures — and struggled most with visually similar ones. Fine-leafed herbs like Mint, Basil, Lagundi, and cilantro kept getting mixed up, suggesting the model hasn't picked up on subtle differences like leaf serration or stem structure. That said, errors were rarely random — wrong predictions usually landed on visually similar classes, not completely unrelated ones. That's actually a good sign. The model has learned something meaningful; it just needs more data, better augmentation, or longer training to separate those tough look-alike pairs.

D. ROC and AUC

9. Which model had the highest AUC score?

MobileNetV2 again. Its ROC curves hugged the top-left corner most consistently across all 20 classes, with standouts like Bay Leaf and Oregano approaching a perfect 1.0. ResNet50 held up on easier classes but dropped off on similar-looking herbs like Mint and cilantro. EfficientNetB0 had the weakest AUC overall — consistent with its blank Grad-CAM heatmaps and lower F1-scores.

10. What does AUC tell us about model performance?

AUC measures how well the model ranks correct predictions above incorrect ones — across every possible threshold, not just one fixed cutoff. A score of 1.0 means perfect separation; 0.5 means random guessing. It's especially useful here because it handles class imbalance well, where accuracy would fall short. A strong macro-average AUC means the model genuinely learned to distinguish herbs — not just memorize the most common ones.

E. Explainability (Grad-CAM)

11. What did Grad-CAM reveal about model decision-making?

It showed what each model actually focused on when predicting. Some highlighted the leaf correctly; others activated on background regions — meaning decent accuracy but for the wrong reasons.

12. Did the model focus on relevant image regions?

Partially. MobileNetV2 and ResNet50 mostly focused on the leaf area, but struggled on trickier classes like Mint and cilantro. EfficientNetB0 was almost entirely dark — not focusing on anything useful.

13. Which model produced the most meaningful heatmaps?

MobileNetV2. Its activations consistently landed on the right regions. ResNet50 was decent but inconsistent. EfficientNetB0 barely produced any visible heatmap, which matches its weaker metrics — it wasn't just underperforming, it wasn't looking at the right things either.

F. Model Comparison & Improvement

14. Which model would you recommend for deployment?

MobileNetV2, without hesitation. It topped every metric — accuracy, F1-score, AUC — and its Grad-CAM heatmaps showed it was focusing on the right regions. It's also lightweight enough to run efficiently in a real app. The other two simply couldn't keep up, especially on visually similar herb classes.

15. How can you further improve it?

A few things would help — more training data per class, stronger augmentation (flips, color jitter, random crops), and unfreezing some of the base layers for fine-tuning. Training for more epochs with a learning rate scheduler would also give it more room to improve. If you want to push further, an ensemble with EfficientNetB0 (once better trained) could help cover each model's weak spots.

G. Real-World Application

16. How can your model be applied in real-world scenarios?

The most obvious use case is a plant identification app — point your camera at an herb and get an instant ID. Beyond that, it could help farmers monitor crops, assist pharmacists or herbalists in verifying medicinal plants, or even be used in schools as an educational tool for learning local flora.

17. What are the risks of deploying an inaccurate model?

Honestly, the stakes can be high. Misidentifying medicinal herbs could lead to wrong remedies or even harmful ones. In farming, a wrong classification could mean treating the wrong plant or ignoring a diseased one. Trust is also a risk — one bad prediction in a real app can shake user confidence entirely. That's why accuracy alone isn't enough; reliability across all classes matters.

18. How can this system be integrated into a mobile/web app?

The trained model can be exported and served through a backend API — a user uploads or captures an image, it gets sent to the server, and the prediction comes back in seconds. For mobile, TensorFlow Lite makes it possible to run the model directly on-device without needing internet. A simple, clean UI with the predicted herb name, confidence score, and maybe a brief description is all you really need to make it useful.